

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory work No. 3

The Command Line: Shell, Pipes and Text Filters

Rotaru Anastasia, FAF-242

Course: **Operating Systems**

Checked by: Patricia Reițman

Chişinău, 2026

Contents

1 Goal of the work **2**

2 Part 1. Move fast, find help **2**
2.1 Observations 2

3 Part 2. What the shell does to your line **3**
3.1 Predictions and outputs 3
3.2 Observations 3

4 Part 3. Redirection, chaining and jobs **4**
4.1 Observations 4

5 Part 4. Answer questions with one-liners **4**
5.1 Observations 5

6 Part 5. Your first script **5**
6.1 Observations 6

7 Conclusion **6**

1 Goal of the work

The purpose of this laboratory work is to navigate the Bash shell efficiently, predict shell expansions and command executions, redirect input and output streams, and answer real questions about data using small text filters joined by pipelines. The work was carried out on the Linux virtual machine `LinuxLabs`, and every part includes the commands I typed together with the output I obtained.

2 Part 1. Move fast, find help

```
nastea@LinuxLabs:/$ type cd; type ls; type echo; which ls
cd is a shell builtin
ls is aliased to ‘ls --color=auto’
echo is a shell builtin
/usr/bin/ls

nastea@LinuxLabs:/$ echo $PATH | tr ’:’ ’\n’
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/usr/games
/usr/local/games
/snap/bin

nastea@LinuxLabs:/$ nosuchprog; echo "exit code: $?"
nosuchprog: command not found
exit code: 127
```

2.1 Observations

1. **Built-ins vs externals:** `cd` and `echo` are shell built-ins, while `ls` is an external command. `cd` must be a built-in because a child process cannot change the working directory of its parent process (the shell).

2. **PATH variable:** There are 9 directories listed in my `PATH`. The `ls` executable is located in `/usr/bin/ls`.

3. **Exit codes:** A command that is not found returns the standard exit code `127`.

4. **Manual mapping:** According to the `man ls` page, the option that sorts by modification time is `-t`.

3 Part 2. What the shell does to your line

3.1 Predictions and outputs

Command	My prediction	Real output	Explanation (if different)
`echo *.c`	a.c b.c	a.c b.c	Shell expands the wildcard to matching files.
`echo file?.txt`	file1.txt file2.txt	file1.txt file2.txt	`?` matches exactly one character.
`echo [a-b]*`	a.c b.c	a.c alfa beta.txt b.c	`*` matches the remainder of strings starting with a or b.
`echo *.xyz`	error / nothing	*.xyz	If no match exists, the literal string is passed.
`ls alfa beta.txt`	'alfa beta.txt'	cannot access 'alfa'...	Unescaped spaces act as argument separators.
`ls "alfa beta.txt"`	'alfa beta.txt'	'alfa beta.txt'	Quotes treat the string as a single argument.
`echo $HOME "$HOME"`	/home/vitalie twice	/home/vitalie /home/vitalie	Double quotes still evaluate variables.
`echo '$HOME' \$HOME`	literal text	$HOME $HOME	Single quotes and backslashes escape variables.
`echo {1..5}; echo $((7*6))`	1 2 3 4 5 and 42	1 2 3 4 5 42	Brace expansion and arithmetic evaluation.

3.2 Observations

1. **Globbing:** The shell (Bash) itself produced the file names. It matches the wildcard `*.c` to the actual files in the directory and replaces the wildcard with their names before passing them as arguments to `echo`.

2. **Fixing `ls`:** The command failed because the space split the input into two distinct arguments (`alfa` and `beta.txt`). There are two ways to fix it: using quotes (`ls "alfa beta.txt"`) or escaping the space (`ls alfa\ beta.txt`).

3. **Quotes:** Double quotes allow variable expansion (such as `$HOME`), whereas single quotes treat everything literally. Modern `ls` wraps output containing spaces in single quotes to show that it is a single file name.

4 Part 3. Redirection, chaining and jobs

```
nastea@LinuxLabs:~/os-lab3$ ls /etc/nope > out.txt 2> err.txt
nastea@LinuxLabs:~/os-lab3$ wc -l out.txt; cat err.txt
0 out.txt
ls: cannot access '/etc/nope': No such file or directory

nastea@LinuxLabs:~/os-lab3$ wc -l < access.log
500

nastea@LinuxLabs:~/os-lab3$ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2

nastea@LinuxLabs:~/os-lab3$ ls /etc/nope > all.txt 2>&1
vitalie@LinuxLabs:~/os-lab3$ ls /etc/nope 2>&1 > all.txt
ls: cannot access '/etc/nope': No such file or directory
```

4.1 Observations

1. **Redirection targets:** The file `err.txt` received the error message (stderr redirected with `2>`), while `out.txt` received the standard output, which was empty.

2. **Order of redirection:** The shell reads redirections from left to right. `> all.txt 2>&1` sends stdout to the file and then points stderr to wherever stdout currently goes (the file). Reversing the order, `2>&1 > all.txt`, points stderr to the screen (the original destination of stdout) first and only then redirects stdout to the file, so the error stays on the screen.

3. **Standard input:** `wc -l < access.log` feeds the file through standard input, so `wc` receives the text but does not know the file name. Passing the file as an argument (`wc -l access.log`) gives `wc` the file name, so it can print it.

4. **Operators:**

   - `;` runs the next command sequentially, regardless of the success or failure of the first one.
   - `&&` (AND) runs the next command *only* if the previous command succeeded (exit code 0).
   - `||` (OR) runs the next command *only* if the previous command failed (non-zero exit code).

5 Part 4. Answer questions with one-liners

```
nastea@LinuxLabs:~/os-lab3$ bash oneliners.sh
100
    125 /page0
    125 /page1
    125 /page2
    125 /page3
    125 /page3
    125 /page2
    125 /page1
    125 /page0
     15 user5
Total requests by user3: 72 | Failed: 15
10:10 user0
10:15 user5
10:20 user3
     40 /usr/sbin/nologin
      6 /bin/false
      2 /bin/bash
      1 /bin/sync
Bonus - Failed at exact 10 minutes:
50
```

5.1 Observations

1. Why does `ls -l /etc | wc -l` return one more line than `ls /etc | wc -l`?
The `-l` (long listing) format prints a `total` summary line (for example `total 128`) before the list of files. The plain `ls` does not print this line, so the count is one higher with `-l`.

6 Part 5. Your first script

```
nastea@LinuxLabs:~/os-lab3$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures

nastea@LinuxLabs:~/os-lab3$ ./report.sh nope.log; echo "exit code: $?"
no such file: nope.log
exit code: 1

nastea@LinuxLabs:~/os-lab3$ cat ~/os-lab3/report.log
500 requests, 100 failed
user0: 14 failures
...
```

6.1 Observations

1. **Shebang:** The first line `#!/bin/bash` tells the operating system to use the Bash interpreter to run the script. Running the script before making it executable with `chmod +x` gives a "Permission denied" error.

2. **Pathing:** The script must be called as `./report.sh` to tell the shell to look in the current directory (`.`). Otherwise the shell searches only the directories listed in `$PATH` and returns "command not found".

3. **Error handling:** The script uses `>&2` to send the error message to standard error (stderr) instead of standard output. It ends with `exit 1` to signal to the operating system and to parent processes that it terminated in a failure state.

7 Conclusion

In this laboratory work I worked with the Bash shell on Linux. I learned how the shell interprets a command line through globbing and expansion before execution. I used input and output redirection to manage normal output and errors, and chained commands with the logical operators `;`, `&&` and `||`. By building pipelines with `grep`, `cut`, `sort` and `uniq`, I extracted and analysed data from log files. Finally, turning these commands into a parameterised, executable Bash script consolidated my understanding of shell scripting, exit codes and automated task scheduling.